

你是一名经验丰富的全栈开发工程师和 UI/UX 设计师。请直接在当前项目目录中，为我开发一个可以正式上线运行的完整网站，不要只提供示例代码、伪代码或开发思路。

一、网站基本信息

网站名称：【FY的小站】

正式域名：【<https://pzq1688.com/>】

网站主题：【资源收藏、学习资料、技术文章、软件下载于一体】

网站语言：默认简体中文。

网站需要能够部署到公网，所有人通过正式域名访问。

整体风格要求：

1. 简洁、现代、耐看，不使用过多花哨动画。
2. 页面留白合理，卡片布局整齐。
3. 支持电脑、手机和平板，自适应不同屏幕。
4. 首页信息丰富，但不能显得杂乱。
5. 按钮、表单、弹窗、加载状态、空状态和错误提示风格统一。
6. 支持浅色模式和深色模式。
7. 优先保证操作简单、加载速度快和移动端体验。
8. 界面使用中文，不出现没有翻译的英文占位内容。

二、技术方案

请使用适合长期维护的主流技术栈，并使用各依赖的最新稳定版本：

- Next.js, 使用 App Router
- TypeScript
- Tailwind CSS
- PostgreSQL
- Prisma ORM
- 成熟可靠的身份认证方案
- Zod 或类似工具进行服务端数据验证
- Docker 和 Docker Compose
- ESLint 和代码格式化工具

所有密钥、数据库地址、管理员初始化信息必须通过环境变量配置，不允许硬编码在代码中。

请提供：

- `.env.example`
- 数据库结构和迁移文件
- 初始化数据脚本
- Dockerfile
- docker-compose.yml

- README.md
- 本地运行说明
- 正式部署说明
- 域名绑定说明
- 数据备份和恢复说明

三、用户身份和权限

网站设置三种身份：

1. 未登录游客

允许：

- 浏览首页
- 浏览公开内容
- 查看分类、标签和详情页
- 搜索内容
- 查看热门内容和最新内容
- 使用浏览器本地临时收藏
- 注册和登录

不允许：

- 进入管理员后台
- 发布、修改或删除网站内容
- 管理其他用户
- 查看其他人的收藏
- 调用管理员接口

2. 注册用户

允许：

- 拥有游客的所有权限
- 将收藏保存到数据库
- 跨设备同步收藏
- 查看“我的收藏”
- 取消收藏
- 修改昵称、头像和密码
- 查看个人资料
- 退出登录
- 注销自己的账号

用户登录后，将当前浏览器里的临时收藏合并到账号收藏中，并自动去重。

3. 管理员

允许：

- 进入 `/admin` 管理后台

- 查看网站数据概览
- 创建、编辑、删除、发布和下架内容
- 管理分类、标签、封面图和推荐位
- 设置内容是否置顶、推荐或隐藏
- 查看和管理用户
- 禁用或恢复用户
- 查看用户收藏数量
- 管理首页轮播图和公告
- 修改网站基础设置
- 查看操作日志

所有管理员权限必须在服务器端验证，不能只通过前端隐藏按钮实现。

普通用户手动访问管理员地址或管理员接口时，必须返回无权限提示或 403，不能泄露后台数据。

四、核心页面

至少实现以下页面：

1. 首页

首页包含：

- 顶部导航栏
- 网站 Logo 和名称
- 搜索框
- 登录、注册和用户头像入口
- 首页轮播图或重点推荐区域
- 分类导航
- 推荐内容
- 最新内容
- 热门内容
- 收藏数量展示
- 网站公告
- 页脚
- 联系方式、隐私政策和用户协议入口

移动端使用适合手机操作的菜单。

2. 内容列表页

功能包括：

- 分类筛选
- 标签筛选
- 关键词搜索
- 最新、最热、推荐排序
- 分页或无限加载
- 卡片和列表两种显示方式
- 加载骨架屏
- 无数据提示

- 请求失败后的重新加载按钮

3. 内容详情页

展示：

- 标题
- 封面图
- 简介
- 正文内容
- 分类和标签
- 发布时间
- 更新时间
- 浏览量
- 收藏量
- 收藏和取消收藏按钮
- 复制链接按钮
- 相关推荐
- 上一篇和下一篇内容

如果内容不存在、已经下架或没有发布，普通用户不能查看。

4. 搜索页面

要求：

- 支持标题和简介关键词搜索
- 支持分类、标签和时间筛选
- 对输入内容进行防抖处理
- 显示搜索结果数量
- 记录热门搜索词
- 没有结果时给出友好提示

5. 我的收藏

要求：

- 只有登录用户可以查看数据库收藏
- 支持按照收藏时间排序
- 支持关键词搜索
- 支持分类筛选
- 支持取消单个收藏
- 支持批量取消收藏
- 支持清空收藏，操作前二次确认
- 收藏内容被删除或下架时显示对应状态

6. 登录和注册

实现：

- 邮箱和密码注册
- 邮箱和密码登录

- 记住登录状态
- 退出登录
- 忘记密码和重置密码的基础结构
- 登录失败次数限制
- 密码显示和隐藏
- 表单实时验证
- 友好的错误提示

密码必须经过安全哈希处理，数据库中不得保存明文密码。

7. 用户中心

包含：

- 用户头像
- 昵称
- 邮箱
- 注册时间
- 收藏数量
- 修改个人资料
- 修改密码
- 退出登录
- 注销账号

修改密码时需要验证当前密码。

8. 管理员后台

后台需要独立布局，至少包含：

- 数据概览
- 内容管理
- 分类管理
- 标签管理
- 用户管理
- 轮播图管理
- 公告管理
- 网站设置
- 操作日志

数据概览显示：

- 用户总数
- 内容总数
- 已发布数量
- 草稿数量
- 总收藏次数
- 今日新增用户
- 今日浏览量
- 热门内容排行

内容管理支持：

- 创建内容
- 编辑内容
- 删除内容
- 草稿保存
- 发布和下架
- 批量操作
- 搜索和筛选
- 设置推荐
- 设置置顶
- 上传封面
- 预览内容

删除内容必须二次确认。

五、数据库设计

请至少设计以下数据表：

- User：用户
- Account：第三方账号预留
- Session：登录会话
- Content：网站内容
- Category：分类
- Tag：标签
- ContentTag：内容与标签关系
- Favorite：用户收藏
- Banner：首页轮播图
- Announcement：网站公告
- SiteSetting：网站设置
- OperationLog：管理员操作日志
- PasswordResetToken：密码重置令牌

用户表至少包含：

- id
- email
- passwordHash
- nickname
- avatar
- role
- status
- createdAt
- updatedAt

收藏表必须设置用户和内容的联合唯一约束，防止同一个用户重复收藏同一条内容。

内容表至少包含：

- id

- title
- slug
- summary
- content
- coverImage
- status
- isFeatured
- isPinned
- viewCount
- favoriteCount
- categoryId
- publishedAt
- createdAt
- updatedAt

删除内容、用户或分类时，需要正确处理关联数据，不能产生无效数据。

六、收藏功能详细要求

收藏功能是网站的重要功能，必须完整实现：

1. 未登录游客点击收藏时，保存到 localStorage。
2. 已登录用户点击收藏时，通过服务端接口保存到数据库。
3. 同一个用户不能重复收藏同一内容。
4. 用户登录后，把本地收藏合并到数据库。
5. 合并完成后清理本地已经同步的收藏。
6. 收藏按钮点击后立即显示状态变化。
7. 如果请求失败，需要恢复原状态并显示错误提示。
8. 收藏和取消收藏接口必须进行身份验证。
9. 不能相信客户端提交的用户 ID，用户 ID 必须从安全会话中获取。
10. 内容被删除或下架后，收藏页面需要正确显示。
11. 管理员可以查看内容收藏数量，但不能查看用户密码等敏感信息。
12. 使用数据库事务或安全更新方式，保证收藏数量不会出现负数或重复累加。

七、安全要求

必须实现或预留以下安全措施：

- 服务端角色权限验证
- 密码安全哈希
- 安全 Cookie
- HttpOnly
- SameSite
- CSRF 防护
- XSS 防护
- SQL 注入防护
- 表单数据验证
- 接口访问频率限制
- 登录失败次数限制
- 文件上传类型限制
- 文件上传大小限制

- 防止路径穿越
- 防止普通用户访问管理员接口
- 不在前端暴露数据库密码和服务端密钥
- 生产环境关闭详细错误堆栈
- 管理员操作日志
- 删除、禁用、清空等危险操作二次确认

前端权限控制只负责改善界面体验，真正的权限判断必须在服务端执行。

八、SEO和访问体验

实现：

- 每个页面独立的标题和描述
- Open Graph 分享信息
- sitemap.xml
- robots.txt
- 规范链接 canonical
- 语义化 HTML
- 图片懒加载
- 图片尺寸优化
- 页面缓存策略
- 404 页面
- 403 页面
- 500 错误页面
- 加载页面
- 空状态页面
- 合理的无障碍属性
- 键盘操作支持
- 手机端触摸区域优化

九、部署和域名

我要让所有人通过以下域名访问：

[【https://pzq1688.com/】](https://pzq1688.com/)

请提供两套部署方案：

方案一：云服务器部署

使用：

- Docker Compose
- PostgreSQL
- Nginx 或 Caddy 反向代理
- HTTPS
- 自动启动
- 日志管理
- 数据库持久化

- 数据库定期备份

给出完整部署命令和配置文件。

方案二：托管平台部署

提供适合该项目的托管部署步骤，并说明：

- 如何创建数据库
- 如何配置环境变量
- 如何执行数据库迁移
- 如何绑定自定义域名
- 如何启用 HTTPS
- 如何创建第一个管理员账号

不能只写概念说明，需要给出可执行步骤。

十、管理员初始化

通过环境变量配置初始管理员：

- ADMIN_EMAIL
- ADMIN_PASSWORD
- ADMIN_NAME

执行初始化脚本时：

1. 如果管理员不存在，则创建管理员。
2. 如果管理员已经存在，则不能重复创建。
3. 管理员密码必须哈希后存储。
4. README 中说明首次登录方式。
5. `.env.example` 中不能填写真实密码。

十一、代码质量

要求：

- 使用清晰的目录结构。
- 页面、组件、服务、权限、数据库操作相互分离。
- 避免把所有代码写在一个文件中。
- 公共逻辑抽取为可复用模块。
- TypeScript 不随意使用 `any`。
- 所有接口有统一的成功和错误响应结构。
- 所有异步请求正确处理加载、成功和失败状态。
- 重要代码添加中文注释。
- 删除无用代码和默认演示页面。
- 不留下无法运行的 TODO。
- 不使用虚假的静态数据代替已经要求实现的数据库功能。
- 确保项目可以成功安装、编译、运行和构建。

十二、测试与验收

至少完成以下测试：

- 用户注册
- 用户登录
- 用户退出
- 游客浏览
- 游客本地收藏
- 登录后合并收藏
- 用户添加收藏
- 用户取消收藏
- 防止重复收藏
- 普通用户无法进入后台
- 普通用户无法调用管理员接口
- 管理员创建内容
- 管理员编辑内容
- 管理员发布和下架内容
- 管理员删除内容
- 手机端页面适配
- 404 和错误页面
- 数据库迁移
- 生产环境构建

最终必须确保以下命令可以正常执行：

```
npm install
npm run dev
npm run lint
npm run build
```

Docker 部署也必须可以正常执行：

```
docker compose up -d --build
```

十三、执行方式

请按照以下顺序执行：

1. 检查当前项目目录。
2. 输出简短的项目架构说明。
3. 创建完整项目文件。
4. 配置数据库模型。
5. 完成身份认证和权限系统。
6. 完成前台页面。
7. 完成在线收藏功能。
8. 完成管理员后台。
9. 完成 Docker 和部署配置。

10. 执行数据库迁移。
11. 执行代码检查和生产构建。
12. 修复所有编译错误。
13. 输出最终运行方法、管理员初始化方法和域名绑定方法。

遇到可以自行判断的问题时，请采用安全、简洁、易维护的方案直接继续，不要停留在询问阶段。

不要只告诉我应该怎么开发，请直接创建和修改项目文件，直到网站能够运行。